a. Name of two GenAI programs

Claude and Gemini

b. How you accessed them (e.g., Google query, CoPilot on my computer)

Claude app on my computer, Gemini from Gemini website

c. Prompt used. (You must use the same prompt for both GenAIs.)

“Generate the program according to the specifications. Make sure to provide adequate documentation throughout.” as well as the assignment direction file, the rubric, and the 5 puzzle text files.

d. Generated code from both GenAIs.

```
 *
================================================================================
 * Program Name : EECS 348 Assignment 4 - Recursive Object-Oriented Sudoku
Solver
 * File         : Assignment4.cpp
 *
 * Description  : Reads Sudoku puzzles from text files, prints each
puzzle, and
 *                uses a recursive depth-first search with backtracking to
find
 *                EVERY solution to the puzzle. Each solution is printed;
if the
 *                puzzle has no solution, "No solution found" is printed.
 *
 * Inputs       : puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt,
puzzle5.txt
 *                (read from the current working directory by default), OR
any
 *                puzzle file names given on the command line, e.g.
 *                    ./Assignment4.exe puzzle1.txt puzzle3.txt
 *                File format: 9 rows of 9 whitespace-separated tokens,
where a
 *                token is a digit 1-9 (a given) or '_' (a blank square).
 *
 * Output       : For each puzzle file (to standard output):
 *                  - the puzzle file name
 *                  - the puzzle as read from the file
 *                  - every solution found (numbered), or "No solution
found"
 *                  - the number of solutions and the time spent searching
 *
```

```
 * Collaborators: None
 *
 * Other sources: Claude (Anthropic), accessed through the claude.ai
web/app
 *                interface (GenAI #1 for this assignment). Prompt used:
 *                "Generate the program according to the specifications.
Make
 *                sure to provide adequate documentation throughout."
 *                (with the Assignment 4 instructions PDF, rubric PDF, and
the
 *                five puzzle files attached).
 *
 * Author       : Erick Marte
 * Created      : October 15, 2026
 * Revised      : October 15, 2026
 *
 * Build (EECS Cycle servers):
 *                g++ -std=c++11 -O2 -Wall -Wextra -o Assignment4.exe
Assignment4.cpp
 * Run:
 *                ./Assignment4.exe
 *
================================================================================
==
 */

#include <array>      // std::array for fixed-size 9x9 grid storage
#include <chrono>     // std::chrono for measuring how long each search
takes
#include <fstream>    // std::ifstream for reading puzzle files
#include <iostream>   // std::cout / std::ostream for printing output
#include <string>     // std::string for file names, tokens, and messages
#include <vector>     // std::vector for the list of solutions and file
names

/*
================================================================================
==
 * Class SudokuBoard
 * Source: generated by Claude (Anthropic) via claude.ai.
```

```
 *
 * Holds one 9x9 Sudoku grid plus three sets of bitmasks that record which
 * digits are already used in each row, each column, and each 3x3 box.
 * Bit d (1..9) of a mask is set when digit d is present in that unit, so
the
 * legal candidates for a cell are found with one OR and one NOT instead
of
 * re-scanning 27 cells every time.
 *
=============================================================================
== */
class SudokuBoard {
public:
    static const int SIZE = 9;                  // the grid is 9 cells wide
and tall
    static const int BOX = 3;                   // each block is 3x3 cells
    static const int EMPTY = 0;                 // 0 stores a blank square
internally
    static const unsigned ALL_DIGITS = 0x3FEu; // bits 1..9 set (binary 11
1111 1110)

    SudokuBoard();                                                          //
builds an empty, consistent board
    bool loadFromFile(const std::string& fileName, std::string& error); //
reads a puzzle file into the board
    void print(std::ostream& out) const;                                    //
prints the grid in the input format
    bool isConsistent() const;                                              //
false if the givens break a rule
    bool findMostConstrainedEmptyCell(int& row, int& col,
                                      unsigned& candidates) const;      //
picks the next cell to branch on
    void place(int row, int col, int digit);                                //
writes a digit and marks it used
    void clear(int row, int col);                                           //
erases a digit and unmarks it

private:
    static int boxIndex(int row, int col);        // maps (row, col) to
its box number 0..8
```

```
    static unsigned bitFor(int digit);            // converts a digit 1..9
to its mask bit
    static int countBits(unsigned mask);          // counts how many
candidates a mask holds

    std::array<std::array<int, SIZE>, SIZE> cells_; // cells_[r][c] holds
0 (blank) or 1..9
    std::array<unsigned, SIZE> rowUsed_;           // rowUsed_[r] =
digits already in row r
    std::array<unsigned, SIZE> colUsed_;           // colUsed_[c] =
digits already in column c
    std::array<unsigned, SIZE> boxUsed_;           // boxUsed_[b] =
digits already in box b
    bool consistent_;                              // true while no given
repeats in a unit
};

/*
-------------------------------------------------------------------------
-
 * Constructor: start with every cell blank and every mask empty.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
SudokuBoard::SudokuBoard() : consistent_(true) {   // a blank board breaks
no rules
    for (int r = 0; r < SIZE; ++r) {               // visit each row
        cells_[r].fill(EMPTY);                     // set every cell in
the row to blank
    }
    rowUsed_.fill(0u);                             // no digits used in
any row yet
    colUsed_.fill(0u);                             // no digits used in
any column yet
    boxUsed_.fill(0u);                             // no digits used in
any box yet
}
```

```
/*
-------------------------------------------------------------------------
-
 * loadFromFile: read exactly 81 tokens ('_' or '1'..'9') from fileName.
 * Returns false and fills `error` if the file is missing or malformed.
 * If a given digit repeats in a row/column/box, the board still loads (so
it
 * can be printed) but is marked inconsistent, meaning it has no solution.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
bool SudokuBoard::loadFromFile(const std::string& fileName, std::string&
error) {
    *this = SudokuBoard();                              // reset to a
blank board before loading
    std::ifstream in(fileName.c_str());                 // open the puzzle
file for reading
    if (!in) {                                          // the file could
not be opened
        error = "could not open file '" + fileName + "'"; // explain the
problem to the caller
        return false;                                   // report failure
    }

    std::string token;                                  // holds one
whitespace-separated token
    int count = 0;                                      // number of cells
read so far (0..81)
    while (in >> token) {                               // read tokens
until end of file
        if (count >= SIZE * SIZE) {                     // more than 81
tokens were found
            error = "more than 81 cells in '" + fileName + "'"; //
describe the format error
            return false;                               // report failure
        }
        int row = count / SIZE;                         // row of this
cell (0..8)
```

```
        int col = count % SIZE;                          // column of this
cell (0..8)

        if (token == "_") {                              // an underscore
marks a blank square
            cells_[row][col] = EMPTY;                    // store it as
blank
        } else if (token.size() == 1 && token[0] >= '1' && token[0] <=
'9') { // a single digit 1-9
            int digit = token[0] - '0';                  // convert the
character to its integer value
            unsigned bit = bitFor(digit);                // mask bit that
represents this digit
            int b = boxIndex(row, col);                  // box that
contains this cell
            if ((rowUsed_[row] & bit) || (colUsed_[col] & bit) ||
(boxUsed_[b] & bit)) { // digit repeats
                consistent_ = false;                     // givens break a
rule -> unsolvable
            }
            cells_[row][col] = digit;                    // store the given
digit
            rowUsed_[row] |= bit;                        // mark the digit
as used in its row
            colUsed_[col] |= bit;                        // mark the digit
as used in its column
            boxUsed_[b] |= bit;                          // mark the digit
as used in its box
        } else {                                         // anything else
is not allowed
            error = "invalid token '" + token + "' in '" + fileName + "'";
// describe the bad token
            return false;                                // report failure
        }
        ++count;                                         // one more cell
has been read
    }

    if (count != SIZE * SIZE) {                          // fewer than 81
cells were found
```

```
        error = "expected 81 cells but found " + std::to_string(count) +
                " in '" + fileName + "'";               // describe the
short file
        return false;                                   // report failure
    }
    return true;                                        // the file loaded
successfully
}

/*
-------------------------------------------------------------------------
-
 * print: write the grid using the same layout as the puzzle files
 * (digits separated by spaces, '_' for blanks, one row per line).
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
void SudokuBoard::print(std::ostream& out) const {
    for (int r = 0; r < SIZE; ++r) {                    // visit each row
        for (int c = 0; c < SIZE; ++c) {                // visit each
column in the row
            if (c > 0) {                                // every cell
except the first in a row
                out << ' ';                             // gets a space
before it
            }
            if (cells_[r][c] == EMPTY) {                // a blank square
                out << '_';                             // is printed as
an underscore
            } else {                                    // a filled square
                out << cells_[r][c];                    // is printed as
its digit
            }
        }
        out << '\n';                                    // end the row
    }
}
```

```
/*
-------------------------------------------------------------------------
-
 * isConsistent: true when no given digit was duplicated in any unit.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
bool SudokuBoard::isConsistent() const {
    return consistent_;                                 // value recorded
while loading
}

/*
-------------------------------------------------------------------------
-
 * findMostConstrainedEmptyCell: scan the blank cells and choose the one
with
 * the FEWEST legal candidates (the "minimum remaining values" rule).
Branching
 * on the tightest cell keeps the search tree small.
 *   - Returns false when no blank cell remains (the board is a solution).
 *   - Returns true with candidates == 0 when some blank cell has no legal
 *     digit (a dead end), which tells the solver to backtrack
immediately.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
bool SudokuBoard::findMostConstrainedEmptyCell(int& row, int& col,
                                               unsigned& candidates) const
{
    bool found = false;                                 // no blank cell
seen yet
    int bestCount = SIZE + 1;                           // larger than any
real candidate count
    for (int r = 0; r < SIZE; ++r) {                    // visit each row
        for (int c = 0; c < SIZE; ++c) {                // visit each
column
```

```
            if (cells_[r][c] != EMPTY) {                // skip cells that
are already filled
                continue;                               // move on to the
next cell
            }
            unsigned used = rowUsed_[r] | colUsed_[c] |
boxUsed_[boxIndex(r, c)]; // digits blocked here
            unsigned options = ALL_DIGITS & ~used;      // digits still
legal for this cell
            int n = countBits(options);                 // how many legal
digits there are
            if (n < bestCount) {                        // this cell is
tighter than the best so far
                found = true;                           // remember that a
blank cell exists
                bestCount = n;                          // record its
candidate count
                row = r;                                // record its row
                col = c;                                // record its
column
                candidates = options;                   // record its
candidate digits
                if (n <= 1) {                           // 0 = dead end, 1
= forced move
                    return true;                        // nothing can be
tighter, stop scanning
                }
            }
        }
    }
    return found;                                       // false means the
grid is full
}

/*
-------------------------------------------------------------------------
-
 * place: put `digit` into (row, col) and mark it used in that
row/col/box.
 * The solver only calls this with a digit that is a legal candidate.
```

```
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
void SudokuBoard::place(int row, int col, int digit) {
    unsigned bit = bitFor(digit);                          // mask bit for
this digit
    cells_[row][col] = digit;                              // write the digit
into the grid
    rowUsed_[row] |= bit;                                  // the row now
contains this digit
    colUsed_[col] |= bit;                                  // the column now
contains this digit
    boxUsed_[boxIndex(row, col)] |= bit;                   // the box now
contains this digit
}

/*
-------------------------------------------------------------------------
-
 * clear: undo a previous place() at (row, col) so the solver can
backtrack.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
void SudokuBoard::clear(int row, int col) {
    unsigned bit = bitFor(cells_[row][col]);               // mask bit of the
digit being removed
    cells_[row][col] = EMPTY;                              // blank the cell
again
    rowUsed_[row] &= ~bit;                                 // the row no
longer contains the digit
    colUsed_[col] &= ~bit;                                 // the column no
longer contains the digit
    boxUsed_[boxIndex(row, col)] &= ~bit;                  // the box no
longer contains the digit
}
```

```cpp
/*
--------------------------------------------------------------------------
-
 * Small static helpers.
 * Source: Claude (Anthropic).
 *
--------------------------------------------------------------------------
- */
int SudokuBoard::boxIndex(int row, int col) {
    return (row / BOX) * BOX + (col / BOX);             // boxes numbered
0..8 left-to-right, top-to-bottom
}

unsigned SudokuBoard::bitFor(int digit) {
    return 1u << digit;                                 // digit d lives
in bit d of a mask
}

int SudokuBoard::countBits(unsigned mask) {
    int n = 0;                                          // running count
of set bits
    while (mask != 0u) {                                // until every set
bit is removed
        mask &= mask - 1u;                              // clear the
lowest set bit
        ++n;                                            // and count it
    }
    return n;                                           // number of bits
that were set
}

/*
==========================================================================
==
 * Class SudokuSolver
 * Source: generated by Claude (Anthropic) via claude.ai.
 *
 * Finds ALL solutions of a puzzle using recursive depth-first search with
 * backtracking, exactly as described in the assignment:
 *   1. pick an empty cell and work out its candidate digits;
```

```
 *   2. if there are none, this path is a dead end -> return (backtrack);
 *   3. otherwise try each candidate in turn and recurse with one more
cell
 *      filled; undo the move afterwards and try the next candidate;
 *   4. when no empty cell is left (k == 81) the grid is a solution ->
save it.
 * Because step 3 keeps going after a success, every solution is found.
 *
================================================================================
== */
class SudokuSolver {
public:
    explicit SudokuSolver(const SudokuBoard& puzzle);   // copies the
puzzle to solve
    void solve();                                       // runs the full
search
    const std::vector<SudokuBoard>& solutions() const;  // every solution
found by solve()

private:
    void search();                                      // the recursive
backtracking step

    SudokuBoard work_;                                  // board that is
filled/unfilled during search
    std::vector<SudokuBoard> solutions_;                // completed grids
collected so far
};

/*
--------------------------------------------------------------------------------
-
 * Constructor: keep a private working copy so the caller's puzzle is
untouched.
 * Source: Claude (Anthropic).
 *
--------------------------------------------------------------------------------
- */
SudokuSolver::SudokuSolver(const SudokuBoard& puzzle) : work_(puzzle) {}
// copy the puzzle
```

```
/*
-----------------------------------------------------------------------------
-
 * solve: clear old results, then start the recursion if the givens are
legal.
 * Source: Claude (Anthropic).
 *
-----------------------------------------------------------------------------
- */
void SudokuSolver::solve() {
    solutions_.clear();                                 // forget results
from any earlier run
    if (!work_.isConsistent()) {                        // givens already
break a Sudoku rule
        return;                                         // so no solution
can exist
    }
    search();                                           // begin the
depth-first search
}

/*
-----------------------------------------------------------------------------
-
 * search: one level of the recursive depth-first search with
backtracking.
 * Recursion depth is at most the number of blank cells (<= 81).
 * Source: Claude (Anthropic).
 *
-----------------------------------------------------------------------------
- */
void SudokuSolver::search() {
    int row = 0;                                        // row of the cell
to branch on
    int col = 0;                                        // column of the
cell to branch on
    unsigned candidates = 0u;                           // legal digits
for that cell
```

```
    if (!work_.findMostConstrainedEmptyCell(row, col, candidates)) { // no
blank cell is left
        solutions_.push_back(work_);                    // the full grid
is a solution: save a copy
        return;                                         // go back and
look for more solutions
    }

    // If candidates == 0 the loop below runs zero times, so this call
simply
    // returns: that is the "no-solution-found" result that triggers
backtracking.
    for (int digit = 1; digit <= SudokuBoard::SIZE; ++digit) { // try
digits 1..9 in order
        if (candidates & (1u << digit)) {               // only digits
that are legal here
            work_.place(row, col, digit);               // make the move
            search();                                   // recurse with
one more cell filled
            work_.clear(row, col);                      // undo the move
(backtrack)
        }
    }
}

/*
-------------------------------------------------------------------------
-
 * solutions: read-only access to the grids found by the last solve().
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
const std::vector<SudokuBoard>& SudokuSolver::solutions() const {
    return solutions_;                                  // hand back the
stored solutions
}
```

```cpp
/*
================================================================================
==
 * Class SudokuApplication
 * Source: generated by Claude (Anthropic) via claude.ai.
 *
 * Drives the program: for each puzzle file it loads the board, prints the
file
 * name and puzzle, runs the solver, and prints every solution (or
 * "No solution found") along with a solution count and the search time.
 *
================================================================================
== */
class SudokuApplication {
public:
    explicit SudokuApplication(const std::vector<std::string>& files); //
stores the files to process
    int run();                                                          //
processes all files

private:
    bool processFile(const std::string& fileName);      // handles a
single puzzle file
    static void printDivider(char ch);                  // prints a
horizontal separator line

    std::vector<std::string> files_;                    // puzzle file
names, in processing order
};

/*
--------------------------------------------------------------------------------
-
 * Constructor: remember which puzzle files to solve.
 * Source: Claude (Anthropic).
 *
--------------------------------------------------------------------------------
- */
SudokuApplication::SudokuApplication(const std::vector<std::string>&
files)
```

```
    : files_(files) {}                                  // copy the list
of file names

/*
-------------------------------------------------------------------------
-
 * run: process every file; return 0 if all loaded, 1 if any failed to
load.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
int SudokuApplication::run() {
    int status = 0;                                     // assume success
until a file fails
    for (std::size_t i = 0; i < files_.size(); ++i) {   // visit each file
name in order
        if (!processFile(files_[i])) {                  // the file could
not be loaded
            status = 1;                                 // remember that
something went wrong
        }
    }
    printDivider('=');                                  // closing line
after the last puzzle
    return status;                                      // becomes the
program's exit code
}

/*
-------------------------------------------------------------------------
-
 * processFile: load, print, solve, and report one puzzle.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
bool SudokuApplication::processFile(const std::string& fileName) {
    printDivider('=');                                  // separate this
puzzle from the previous one
```

```
    std::cout << "Puzzle file: " << fileName << '\n';    // required: the
puzzle file name

    SudokuBoard puzzle;                                  // board that will
hold the file's contents
    std::string error;                                   // receives a
message if loading fails
    if (!puzzle.loadFromFile(fileName, error)) {         // try to read the
puzzle
        std::cout << "Error: " << error << '\n';         // explain why it
could not be read
        return false;                                    // report failure
to run()
    }

    std::cout << "\nPuzzle:\n";                          // label for the
original puzzle
    puzzle.print(std::cout);                             // required: the
puzzle as given in the file

    SudokuSolver solver(puzzle);                         // solver working
on a copy of the puzzle
    std::chrono::steady_clock::time_point start =
        std::chrono::steady_clock::now();                // time just
before the search starts
    solver.solve();                                      // find every
solution
    std::chrono::steady_clock::time_point stop =
        std::chrono::steady_clock::now();                // time just after
the search ends
    double ms = std::chrono::duration<double, std::milli>(stop -
start).count(); // elapsed milliseconds

    const std::vector<SudokuBoard>& found = solver.solutions(); // results
of the search
    if (found.empty()) {                                 // the search
found nothing
        std::cout << "\nNo solution found\n";           // required
message for unsolvable puzzles
```

```
    } else {                                            // at least one
solution exists
        for (std::size_t i = 0; i < found.size(); ++i) { // print each
solution in turn
            std::cout << "\nSolution " << (i + 1) << " of " <<
found.size() << ":\n"; // numbered label
            found[i].print(std::cout);                  // required: the
solved grid
        }
    }

    std::cout << "\nSolutions found: " << found.size()  // total number of
solutions
              << "   (search time: " << ms << " ms)\n"; // how long the
search took
    return true;                                        // this file was
processed successfully
}

/*
-------------------------------------------------------------------------
-
 * printDivider: print a 50-character line of `ch` to separate sections.
 * Source: Claude (Anthropic).
 *
-------------------------------------------------------------------------
- */
void SudokuApplication::printDivider(char ch) {
    std::cout << std::string(50, ch) << '\n';           // 50 copies of
ch, then a newline
}

/*
=========================================================================
==
 * main: choose the puzzle files and run the application.
 * With no arguments the five assignment files are used; otherwise every
 * command-line argument is treated as a puzzle file name.
 * Source: Claude (Anthropic).
```

```
 *
==========================================================================
== */
int main(int argc, char* argv[]) {
    std::vector<std::string> files;                    // puzzle files to
process
    if (argc > 1) {                                    // file names were
given on the command line
        for (int i = 1; i < argc; ++i) {               // visit each
argument after the program name
            files.push_back(argv[i]);                  // add it to the
list
        }
    } else {                                           // no arguments:
use the assignment's files
        files.push_back("puzzle1.txt");                // first assigned
puzzle
        files.push_back("puzzle2.txt");                // second assigned
puzzle
        files.push_back("puzzle3.txt");                // third assigned
puzzle
        files.push_back("puzzle4.txt");                // fourth assigned
puzzle
        files.push_back("puzzle5.txt");                // fifth assigned
puzzle
    }

    SudokuApplication app(files);                      // object that
runs the whole program
    return app.run();                                  // 0 = all files
loaded, 1 = a load failed
}
```

```
/**
 * @file Assignment4.cpp
 * @name EECS 348 Assignment 4 - Object-Oriented Sudoku Solver
 * @brief An object-oriented program that solves partially completed
Sudoku
```

```
 *        puzzles using a recursive backtracking (depth-first search)
algorithm.
 * @inputs puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt, puzzle5.txt
 * @output Puzzle file name, initial puzzle grid, and solved puzzle grid
(or "No solution found")
 * @collaborators None (Authored independently following course
guidelines)
 * @author Erick Marte
 * @creation_date October 15, 2026[cite: 7]
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>

using namespace std;

/**
 * @class SudokuSolver
 * @brief Encapsulates a 9x9 Sudoku grid, file input/output, and recursive
backtracking solver logic.
 * @authorship Authored by Student Name.
 */
class SudokuSolver {
private:
    // 9x9 grid representing the Sudoku board where 0 represents an empty
cell
    int board[9][9];

    /**
     * @brief Checks if placing a number at a given row and column is
valid.
     * @param row The row index (0-8)
     * @param col The column index (0-8)
     * @param num The digit to check (1-9)
     * @return true if valid, false otherwise
     * @authorship Authored by Student Name.
     */
    bool isValid(int row, int col, int num) {
```

```
        // Loop through each column in the current row to check for
duplicates
        for (int c = 0; c < 9; c++) {
            if (board[row][c] == num) {
                return false; // Duplicate found in row
            }
        }

        // Loop through each row in the current column to check for
duplicates
        for (int r = 0; r < 9; r++) {
            if (board[r][col] == num) {
                return false; // Duplicate found in column
            }
        }

        // Determine the top-left starting coordinates of the 3x3 sub-grid
(block)
        int startRow = row - (row % 3);
        int startCol = col - (col % 3);

        // Loop through the 3x3 block to check for duplicates
        for (int r = 0; r < 3; r++) {
            for (int c = 0; c < 3; c++) {
                if (board[startRow + r][startCol + c] == num) {
                    return false; // Duplicate found in 3x3 block
                }
            }
        }

        return true; // Position is valid for the given number
    }

    /**
     * @brief Recursive backtracking function to solve the Sudoku puzzle.
     * @return true if a solution is found, false otherwise
     * @authorship Authored by Student Name.
     */
    bool solveRecursive() {
        int row = -1;
```

```
        int col = -1;
        bool isEmpty = false;

        // Scan the board to find the next empty cell (represented by 0)
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == 0) {
                    row = r;
                    col = c;
                    isEmpty = true;
                    break; // Found an empty cell, exit inner loop
                }
            }
            if (isEmpty) {
                break; // Exit outer loop once empty cell is located
            }
        }

        // If no empty cells remain, the puzzle is successfully solved
        if (!isEmpty) {
            return true;
        }

        // Try candidate digits from 1 to 9 for the empty cell
        for (int num = 1; num <= 9; num++) {
            // Check if candidate digit is valid according to Sudoku rules
            if (isValid(row, col, num)) {
                board[row][col] = num; // Temporarily assign candidate
value

                // Recursively attempt to solve the rest of the board
                if (solveRecursive()) {
                    return true; // Solution found
                }

                board[row][col] = 0; // Backtrack if assignment leads to a
dead end
            }
        }
```

```
        return false; // Trigger backtracking if no candidates work
    }

public:
    /**
     * @brief Default constructor initializing the board to zeros.
     * @authorship Authored by Student Name.
     */
    SudokuSolver() {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                board[r][c] = 0;
            }
        }
    }

    /**
     * @brief Loads a Sudoku puzzle from a specified text file.
     * @param filename The path to the puzzle file (e.g.,
puzzle1.txt)[cite: 7]
     * @return true if loaded successfully, false otherwise
     * @authorship Authored by Student Name.
     */
    bool loadPuzzle(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            return false; // Return false if file cannot be opened
        }

        char ch;
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (!(file >> ch)) {
                    return false; // Handle unexpected end of file
                }
                // Convert underscore character to 0, or character digit
to integer value
                if (ch == '_') {
                    board[r][c] = 0;
                } else if (ch >= '1' && ch <= '9') {
```

```
                    board[r][c] = ch - '0';
                } else {
                    board[r][c] = 0; // Default fallback for malformed
input
                }
            }
        }
        file.close();
        return true;
    }

    /**
     * @brief Prints the current state of the Sudoku board to standard
output.
     * @authorship Authored by Student Name.
     */
    void printBoard() const {
        for (int r = 0; r < 9; r++) {
            for (int c = 0; c < 9; c++) {
                if (board[r][c] == 0) {
                    cout << "_ "; // Print underscore for empty cells
                } else {
                    cout << board[r][c] << " ";
                }
            }
            cout << endl; // Newline at the end of each row
        }
    }

    /**
     * @brief Public interface to execute the solving algorithm.
     * @return true if solved, false if no solution exists
     * @authorship Authored by Student Name.
     */
    bool solve() {
        return solveRecursive();
    }
};

/**
```

```
 * @brief Main execution function processing all five required puzzle
files[cite: 7].
 * @return int Exit status code
 * @authorship Authored by Student Name.
 */
int main() {
    // Array containing the five required puzzle filenames[cite: 7]
    string puzzleFiles[5] = {
        "puzzle1.txt",
        "puzzle2.txt",
        "puzzle3.txt",
        "puzzle4.txt",
        "puzzle5.txt"
    };

    // Iterate through each puzzle file sequentially
    for (int i = 0; i < 5; i++) {
        SudokuSolver solver;

        // Output the puzzle file name header[cite: 7]
        cout << "========================================" << endl;
        cout << "Puzzle File: " << puzzleFiles[i] << endl;
        cout << "========================================" << endl;

        // Attempt to load the puzzle from disk
        if (!solver.loadPuzzle(puzzleFiles[i])) {
            cout << "Error: Could not open or read " << puzzleFiles[i] <<
endl;
            continue;
        }

        // Print the initial puzzle loaded from the file[cite: 7]
        cout << "Initial Puzzle:" << endl;
        solver.printBoard();
        cout << "----------------------------------------" << endl;

        // Execute the recursive solver and print the outcome[cite: 7, 9]
        if (solver.solve()) {
            cout << "Solution Found:" << endl;
            solver.printBoard();
```

```
        } else {
            cout << "No solution found" << endl; // Required fallback
message[cite: 7]
        }
        cout << endl;
    }

    return 0;
}
```

e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.

Correctness: Both programs can accurately find a solution (or declare no solution) to a given sudoku puzzle, however, in the case that a puzzle has multiple solutions, like in puzzle5, Gemini fails to find both solutions as it ends once a single solution is found. Additionally Gemini has nothing to handle an incorrect input, so if a given character isn’t a number 1-9 or an underscore, it will still try to solve.

Execution Time: Both programs have similar execution time and time complexity. Due to the structure of the 9x9 sudoku board, the time complexity is O(9^n) with n being the number of open spots that must be recursively solved (and there being 9 “possible” numbers for each spot). This obviously scales up pretty quickly considering there can be up to 81 blank spaces (technically 64 blank spaces ensures a unique solution so it wouldn’t be used to solve for many more than that). While not ideal, this is forced by the logic of the problem.

Claude also has ~double the amount of code as Gemini, so it would take slightly longer to compile (mostly negligible).

Space Complexity: Both programs also have similar space complexities. The board is a constant O(1) complexity → 81 (ints) * 4 (bytes) = 324 bytes. The recursive call stack is O(n) as the program goes to one blank and solves it (with recursive calls) before going to the next blank. Again, n is the number of open spaces in the puzzle. Thus, it doesn’t scale too quickly so space is not an issue. Claude also stores the solution as vectors which takes some memory whereas Gemini does not, simply outputting the solution immediately.

Maintainability: Claude’s structure has 3 classes, each with their own assigned job. Gemini’s has a single “do it all” class which is less intuitive. Both programs have sufficient documentation (lots of inline comments, prologue comments, and function descriptions). It should be noted that Gemini says “@Authorship authored by student name” above every function which is wholly superfluous.

f. Select one to use as the basis for this assignment and justify why you picked it.

The Gemini program as it has the most glaring issues

g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.

Correctness: The program now will output multiple solutions if the board has multiple solutions. Additionally if given incorrect inputs, it will notice and not try to solve an incorrect board.

Execution Time: The time complexity remains unchanged (worst case is still O(9^n), however, the program uses less overall recursive calls because it focuses on spaces with more constraints (spaces where a specific number is “forced”) leading to a faster execution.

Space Complexity: Since there are less recursive calls overall, the call stack will take up less memory on average. As mentioned before, space was a relative non-issue in both programs.

Maintainability: Superfluous comments were removed and documentation and authorship is now clearer.